

TP 2: Récursivité

BUT Informatique - S5 - Qualité algorithmique

1 Fibonacci récursif vs. itératif

La suite de Fibonacci, pour $n \geq 0$ est définie par l'équation de récurrence suivante:

$$F(n) = \begin{cases} n & \text{si } n \leq 1 \\ F(n-1) + F(n-2) & \text{sinon} \end{cases}$$

1.1 Algorithme récursif naïf

Implémentez une méthode récursive

```
public static int fibo_rec_naif(int n);
```

qui calcule le terme de rang n en suivant explicitement la définition précédente. Vous incluez un compteur qui comptabilise uniquement le nombre d'additions réalisées par cette méthode.

1.2 Algorithme itératif

Implémentez la même fonction mais de manière itérative:

```
public static int fibo_iter(int n);
```

Utilisez également un compteur pour comptabiliser uniquement le nombre d'additions réalisées (on ne tiendra pas compte des additions effectuées sur l'indice de boucle pour réaliser les itérations). Comparez les performances des deux méthodes. Que constatez-vous ? Pourquoi la méthode récursive est-elle aussi inefficace ?

1.3 Algorithme récursif efficace

Implémentez une méthode récursive

```
public static int fibo_rec(int n);
```

qui calcule le n -ième terme de la suite de Fibonacci avec le même nombre d'additions que la méthode itérative. Vérifier que ces nombres sont effectivement identiques.

2 Permutations

On appelle permutation d'une chaîne de caractères s toute chaîne de même longueur que s contenant les mêmes caractères que s . Par exemple, pour "abc", les permutations sont "abc", "acb", "bac", "bca", "cab", "cba". Dans cet exercice, on souhaite écrire un programme qui génère toutes les permutations d'une chaîne et étudier sa complexité. On supposera, pour simplifier un peu le problème, que toutes les lettres du mot d'entrée sont distinctes les unes des autres.

2.1 Nombre de permutations $P(n)$

Pour une chaîne s de longueur n et dont tous les caractères sont distincts, combien y a-t-il de permutations de cette chaîne ? Justifiez votre réponse. On notera $P(n)$ le nombre.

2.2 Équation de récurrence de $P(n)$

Établissez une équation de récurrence pour $P(n)$. Comment démontrer que votre réponse à la question précédente est bien solution de cette équation ?

2.3 Complexité asymptotique

Déduisez des questions précédente une borne inférieure de la complexité asymptotique d'un algorithme qui produit toutes les permutations d'une chaîne s .

2.4 Algorithme récursif

On veut générer toutes les permutations de la chaîne $s = c + m$ (où c est le premier

caractère et m la sous-chaîne restante).

Comment générer les permutations de s à partir des permutations de m ?

Indice: Prenez un exemple concret avec $s = \text{"abc"}$: Quelles sont les permutations de $m = \text{"bc"}$? Comment construire les permutations de "abc" à partir de celle de "bc" ?

2.5 Implémentation

Implémenter l'algorithme récursif en suivant l'approche précédente. Vous aurez très probablement à implémenter une fonction auxiliaire qui sera également récursive.

2.6 Complexité asymptotique

Quel est le coût et la complexité asymptotique de votre algorithme ?

3 Arithmétique récursive (exercice bonus)

3.1 Somme

Écrire un algorithme récursif de calcul de la *somme* de deux entiers *naturels* `a` et `b` en supposant que les seules opérations de base dont vous disposez sont

- l'ajout de 1 à un entier `x` : `x+1`
- le retrait de 1 à un entier `x` : `x-1`
- et la comparaison à 0 d'un entier `x` : `a==0`.

Pour ce faire, vous écrirez les méthodes correspondantes suivantes:

```
static public int inc(int x);  
static public int dec(int x);  
static public boolean isZero(int x);  
static public int add(int a, int b);
```

où `add` ne peut donc utiliser aucun opérateur arithmétique en dehors de fonctions `inc` et `dec`.

3.2 Produit

Écrire un algorithme récursif de calcul du *produit* de deux entiers *naturels* `a` et `b` en supposant que les seules opérations de base dont vous disposez sont

- la somme de deux entiers `a` et `b` : `a+b`
- le retrait de 1 à un entier `a` : `a-1`
- et la comparaison à 0 d'un entier `a` : `a==0` .

Vous devez donc implémenter la méthode:

```
static public int mult(int a, int b);
```

3.3 Puissance entière

Écrire un algorithme récursif de calcul de la puissance `n`-ième (`n` entier > 0) d'un nombre réel `a` en supposant que les seules opérations de base dont vous disposez sont

- le produit de deux réels `a` et `b` : `a*b`
- le retrait de 1 à un entier `a` : `a-1`
- et la comparaison à 0 d'un entier `a` : `a==0` .

Vous devez donc implémenter la méthode:

```
static public int puis(int a, int b);
```

C. Lhoussaine